Sebastian Mocanu[1] Sebastian-Ion Nae[1*] Mihai-Eugen Barbu[1*] Marius Leordeanu[1, 2, 3]
[1]National University of Science and Technology POLITEHNICA Bucharest, Romania
[2]Institute of Mathematics "Simion Stoilow" of the Romanian Academy, Romania
[3]NORCE Norwegian Research Center, Norway
{sebastian.mocanu, sebastian_ion.nae.stud.fils, mihai_eugen.barbu.stud.acs}@upb.ro, leordeanu@gmail.com

Abstract

1.7M
IBVS
IBVS IBVS
11×
fiducial
marker GPS
(1) IBVS (2)
YOLOv11 U-Net (3)

1.

GPS IMU 9
[?] [? ?
] GPS
[?] [?
] [?] [?
] [?]
[?
?] IBVS
[?] ArUco [? ? ?]
AprilTag [? ? ?] fiducial
marker
IBVS

YOLO [? ? ?]
fiducial marker IBVS

2.

PBVS IBVS PBVS
[? ?] IBVS [? ?
?] IBVS [? ?]

*These authors contributed equally to this work.

IBVS

$$\mathbf{v} = -\lambda \mathbf{L}_s^{+}(\mathbf{s} - \mathbf{s}^*) \tag{1}$$

$\mathbf{v} = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$ λ $\mathbf{s}$
$\mathbf{s}^*$
$\mathbf{L}_s$

$$\dot{\mathbf{s}} = \mathbf{L}_s \mathbf{v} \tag{2}$$

(u, v) Z

$$\mathbf{L}_s = \begin{pmatrix} -\frac{f}{Z} & 0 & \frac{\bar{u}}{Z} & \frac{\bar{u}\bar{v}}{f} & -\frac{f^2+\bar{u}^2}{f} & \bar{v} \\ 0 & -\frac{f}{Z} & \frac{\bar{v}}{Z} & \frac{f^2+\bar{v}^2}{f} & -\frac{\bar{u}\bar{v}}{f} & -\bar{u} \end{pmatrix} \tag{3}$$

$\bar{u} = u - u_0$ $\bar{v} = v - v_0$ f
$\mathbf{L}_s^{+}$

$$\mathbf{L}_s^{+} = (\mathbf{L}_s^T \mathbf{L}_s)^{-1} \mathbf{L}_s^T \tag{4}$$

IBVS Inria Parrot Bebop 2 ArUco [?] PID [?
] [? ?]

[?]
[?] CNN
[?] YOLO
[? ?] [? ?] [?] [?
] [?]
Liu [?]
[?
]
YOLO IBVS
fiducial marker

3. Our Approach

Our method consists of a teacher-student framework for vision-based quadrotor control, both teacher and

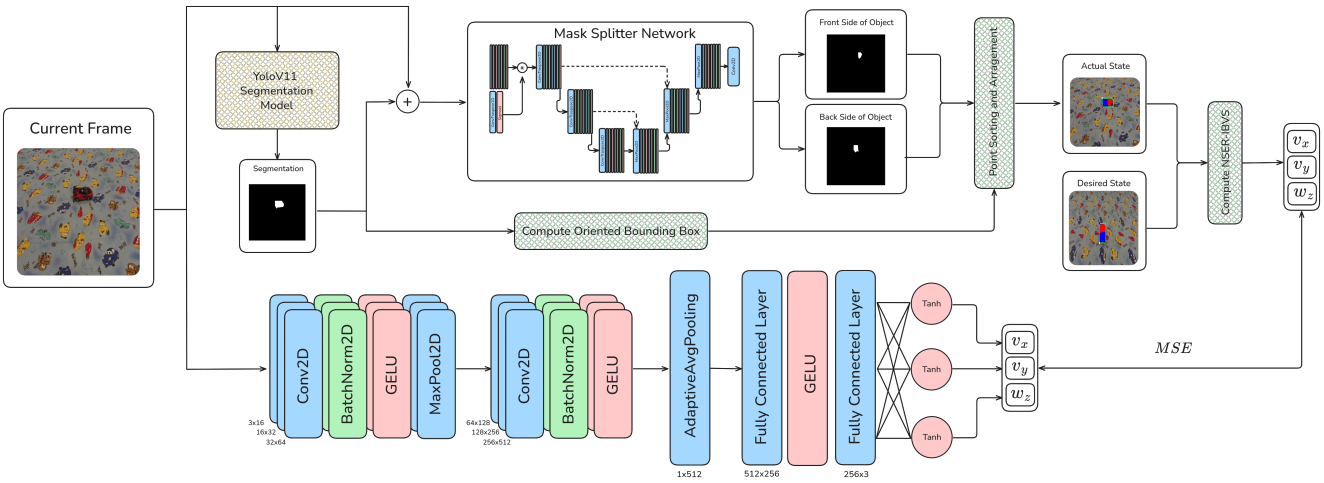


Figure 1. Overview of the proposed Teacher-Student Self-supervised Neuro-Analytic model. Top row illustrates the NSER-IBVS (analytical) Teacher path, which uses YOLO for object segmentation and a mask splitter network to estimate object orientation relative to the drone camera. The bottom one represents the Student path, which learns by minimizing the MSE cost between its output drone commands (ν_x, ν_y, ω_z) and the Teacher's output.

student are tested in a digital-twin of the real-world environment.

The teacher employs a two-stage neural network processing system that is combined with an image-based visual servoing algorithm. In the first stage, a small YOLOv11 Nano (2.84M parameters) instance segmentation network is used. This network is initially trained on synthetic data from the digital-twin and subsequently fine-tuned using real images. This stage has two main objectives: to compute an oriented bounding box (OBB) of the car and to provide its mask to the second stage of our system, as the OBB alone is insufficient for accurate pose inference. The second stage is a specialist neural network trained to split the YOLO masks in half, generating masks that represent the front and back regions of the target vehicle. Knowing the position of these front and back masks and having the OBB, we can rearrange the keypoints to generate four ordered points (two per region) that serve as stable visual features for IBVS. The image Jacobian and control velocities are computed using these features to minimize the error and achieve the desired relative state. The desired orientation keypoints are established by capturing the target pose at the beginning, applying segmentation, and extracting the corresponding feature points. Then we preprocess these keypoints for stability enhancement, detailed in Section 3.1. Our method then rectifies its pose with respect to the desired reference pose.

This framework constitutes the teacher model, which is distilled into a compact convolutional neural network trained via regression to output quadrotor control velocities from camera input RGB images detailed in Section 3.2.

3.1. Numerically Stable Efficient Reduced IBVS

The proposed method begins by training a YOLOv11 Nano instance segmentation model using simulation-generated data, forming the foundation of a two-stage segmentation pipeline. The first stage segments the entire vehicle, while a secondary network splits the segmented region into anterior and posterior components, as illustrated in Figure 2. While effective for segmentation, the first stage generates bounding boxes that may include parts not belonging to the object. To address this issue, we compute a bounding box around the segmented object mask. This produces an unstable point ordering at inference time, introducing inconsistencies for the analytical IBVS algorithm. By determining the positions of the frontal and rear masks, we can rearrange these points to create an oriented bounding box that better represents the car pose and provides more stable feature points, as shown in Figure 3.

The mask splitting network employs a U-Net [?] architecture with attention mechanisms [?], accepting a 4-channel input tensor that comprises the original RGB image and the binary vehicle mask from YOLOv11. The encoder-decoder structure features skip connections and progressively downsamples through three stages with channel dimensions of 32, 64, 128, and 256. An attention mechanism applied to the mask channel generates spatial weights that modulate image features to focus on vehicle regions. The decoder reconstructs spatial resolution through transposed convolu-

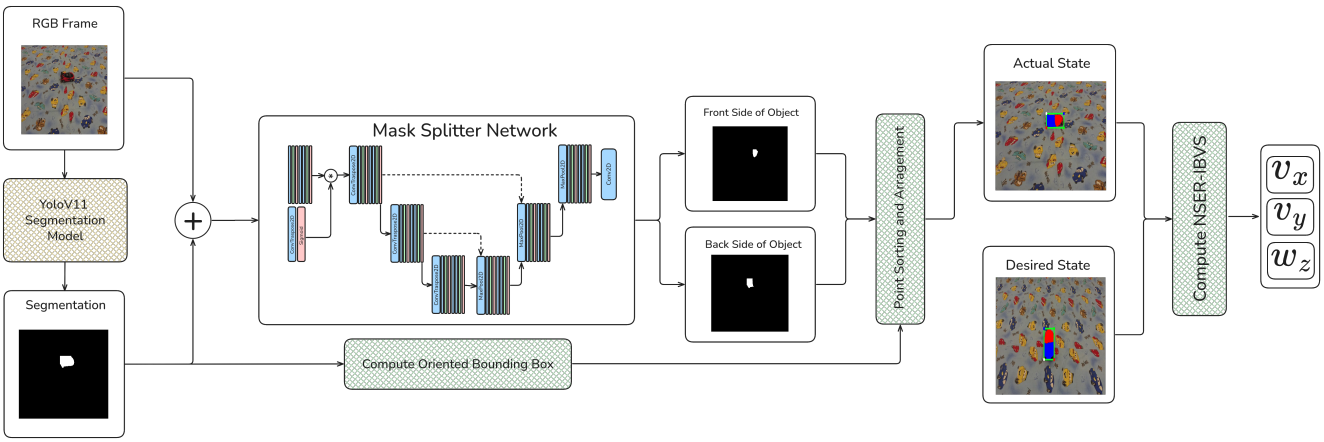


Figure 2. Overview of the proposed Numerically Stable, Efficient, Reduced Image Based Visual Servoing (NSER-IBVS) architecture. The pipeline takes an RGB frame, generates segmentation, and feeds them as input to the Mask Splitter Network. From the original segmentation, bounding box coordinates are determined and used together with the divided front and back masks to rearrange coordinates positions. These coordinates serve as visual features for the analytical model to compute the velocity commands (ν_x, ν_y, ω_z).

Figure 3. Comparison of different bounding box approaches derived from segmentation results: (left) regular bounding box including parts that do not belong to the object, (middle) oriented bounding box that may vary in orientation between frames, and (right) oriented bounding box using a mask-splitting network to separate anterior and posterior vehicle components for improved ordering stability.

tions, producing a 2-channel output that represents anterior and posterior segmentation masks. The network contains approximately 1.94M parameters and incorporates dropout regularization in deeper layers preventing overfit.

The splitter loss function partitions a binary segmentation mask into complementary front and back regions by reconstructing the original input mask. Multiple constraints enforce proper partitioning: individual binary cross-entropy losses ensure that each predicted mask matches its ground truth target, a partition constraint penalizes deviations when the sum of both predicted masks differs from the original mask, and overlap penalties discourage simultaneous occupancy of the same pixels. The coverage loss ensures accurate reconstruction by penalizing both excess predictions beyond the original mask and the missed areas within it. Loss weights are dynamically scheduled during training, emphasizing individual mask accuracy, then gradually shifting toward stronger partition and overlap constraints.

The resulting segmentation masks are processed to compute an oriented bounding box by fitting the minimum enclosing rectangle to the object boundary. The four corner coordinates of this bounding box serve as visual features of the IBVS framework. To ensure keypoint stability and consistent ordering, we employ a preprocessing algorithm that assigns points to anterior or posterior regions based on their proximity to the respective mask centroids. The points are subsequently ordered clockwise to maintain temporal consistency across frames, thereby preventing feature correspondence ambiguity that could destabilize the IBVS control loop.

Control velocity computation is achieved by comparing current keypoint coordinates with reference features obtained from a desired pose configuration. Due to the underactuated nature of the quadrotor platform, we use an efficient approach to compute only the necessary velocity components from the classical IBVS controller by eliminating the angular velocity components ω_x and ω_y and the linear velocity ν_z. Furthermore, since our controller operates exclusively at fixed altitude, we also remove the linear velocity component v_z.

Starting from Equation 3, the reduced interaction

matrix takes the form:

$$\begin{pmatrix} \dot{u} \\ \dot{v} \end{pmatrix} = \begin{pmatrix} -\frac{f}{Z} & 0 & \bar{v} \\ 0 & -\frac{f}{Z} & -\bar{u} \end{pmatrix} \begin{pmatrix} v_x \\ v_y \\ \omega_z \end{pmatrix} \tag{5}$$

that we use to stack the Jacobians corresponding to the keypoints from the oriented rectangle.

3.2. Teacher-Student Model Distillation

To enable real-time deployment, we employ knowledge distillation [?] to transfer the visual servoing capabilities from the teacher IBVS pipeline to a lightweight, low-cost student network. The student network is designed as a 1.7M-parameter CNN that directly regresses control velocities from RGB images using mean squared error (MSE) loss.

A key component of this architecture is the normalization of the target labels (the velocities commands) and their respective de-normalization at inference time. To train this student network, we require both RGB images and their corresponding control velocities. We analyzed the values from the experiments conducted on both real-world and digital-twin environments and determined the minimum and maximum bounds for each command across all scenes, as shown in Table 1. The normalization of target labels improves training stability by ensuring all velocity commands operate within similar numerical ranges, preventing certain outputs from dominating the loss function due to scale differences. This approach also enhances gradient flow during backpropagation and enables the use of bounded activation functions like tanh, which naturally constrains the network outputs to physically meaningful control ranges while maintaining differentiability throughout the training process.

	ν_x		ν_y		ω_z	
	min	max	min	max	min	max
Sim	-24	16	-9	9	-27	40
Real	-30	30	-30	30	-40	40

Table 1. Minimum and maximum command values for translational velocities ν_x and ν_y and rotational velocity ω_z in digital-twin (Sim) and real-world (Real) collected from NSER-IBVS evaluation runs. Used to determine the normalization of the target labels for the self-supervised method.

The student architecture consists of a convolutional feature extractor followed by fully connected regression layers. The convolutional backbone progressively increases channel dimensions from 16 to 512 through six convolutional blocks, each incorporating batch normalization and GELU activation functions. Max pooling is applied after the first two blocks for spatial downsampling, while the final layers use kernel sizes of 3x3 to capture fine-grained spatial features. An adaptive global average pooling layer reduces spatial dimensions to 1x1, followed by two fully connected layers (512→256→3) that output final velocity commands.

Given the limited availability of real-world data, training is performed in two stages. The first stage is pretraining on simulated trajectories from the digital-twin. The second stage is fine-tuning on real-world data. For the real-world fine-tuning stage, we implement targeted data augmentation strategies to enhance dataset diversity and improve generalization to unseen conditions.

The distillation process transfers knowledge from the NSER IBVS teacher to the neural student, enabling direct end-to-end control from visual input while maintaining the characteristics of the classical visual servoing approach, thereby yielding similar trajectories and behavior.

3.3. Data Generation and Simulation Environment

The experimental environment is replicated using a simulator based on the Parrot Anafi drone platform with its accompanying Sphinx simulation framework [?]. This simulation represents a digital-twin for our real-world testing environment, featuring accurate measurements, object structure and poses. The physical test setup consists of a 5×4-meter carpet with a centrally positioned toy car target and the quadrotor positioned at varying poses from the object. The carpet is necessary because the bunker-like environment has a non-Lambertian floor, which poses challenges for our UAV to maintain a fixed position.

Simulated environment creation uses Unreal Engine 4 for level design and asset placement and Blender [?] to create the meshes for most of the assets. Since the target vehicle was harder to reproduce in Blender, it was digitized using Hunyuan3D-2 [?] to generate a .fbx mesh file, which is then scaled to match the dimensions of the physical objects in the real-world for accurate simulation fidelity.

Data generation involves flight sequences within both the digital-twin and real-world environments using varying gimbal camera angles (30°, 45°, 60°, 75°, 90°) for training and 45° for validation, since that is the angle we want to test our method. For the simulated environment, we used two rendering configurations: low and high quality that affect lighting and mesh fidelity to ensure generality and a wide-range of FOVs. With this approach, we created a dataset of 14693 frames for training and 1123 for validation from the digital-twin environment, and 13760 frames

for training and 1084 frames for validation from the real-world dataset. The frames were subsequently augmented to enhance generalization capability.

The collected data serves as input for both the YOLO segmentation and Mask Splitter pipelines employed for object segmentation and keypoint orientation reassignment. For the YOLO model, we annotated the data using polygons over 9302 frames for simulated environment and 8637 for the real one. After training the segmentation model, for the Mask Splitter we implemented a simple labeling tool which divides the car segmentation based on the user interaction. The algorithm first calculates the centroid of the car mask using image moments and then prompts the user to click around the front portion of the vehicle. Using the centroid as reference point, it creates a direction vector from the center to the user-selected front point and projects all mask pixels onto this vector using dot product calculations. Pixels with positive dot products lie in the same direction as the front point relative to the centroid and are assigned to the front mask, while the negative dot products are assigned to the back mask, effectively creating a linear split that separates the vehicle into front and rear segments.

4.

4.1.

8 100 800
4

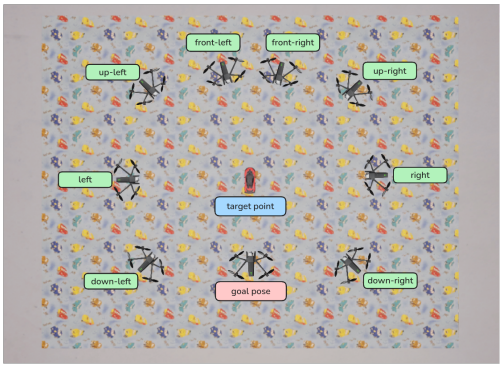


Figure 4. 8
$\pm 45°$
$\pm 14°$

30 240 74307
5 4 + 1 epoch
v_x v_y ω_z
1 224×224 Adam
0.001 3 10^{-4}

4.2.

640 × 360 75
5 ≤ 80 3
0 5 3 ≤ 40
NSER IBVS L2
IBVS YOLOv11 Mask Splitter 3
0 1 5
3 ≤ 1
10 4 80
43963 240

4.3.

3
IoU 45

4.4.

14.261
29.756 52% IoU 0.752
0.522 2
6

NSER IBVS
640 × 360 30
FPS
3

80

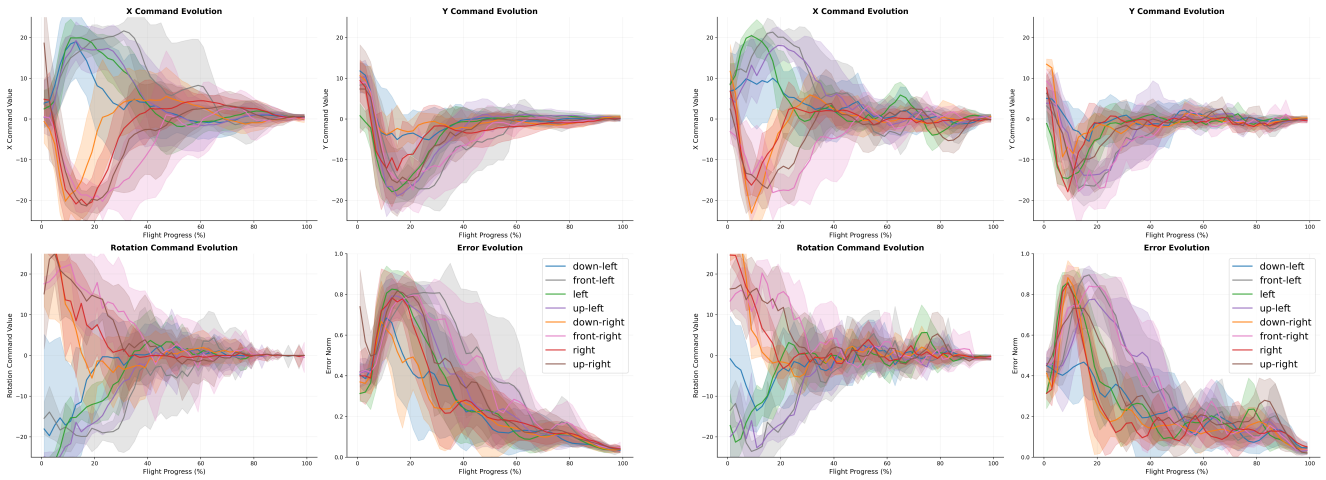
5.

Con-
vNet NSER-
IBVS
1.7M 11× 529.77 FPS
48.3 FPS 80

YOLOv11 U-Net fidu-
cial marker Parrot Anafi

29.956 33.334 IoU 0.627
0.591

”Romanian Hub for Artificial Intelligence - HRIA” Smart Growth Digitization and Financial Instruments Program, 2021-2027 MySMIS 334906 European Health and Digital Executive Agency HADEA DIGITWIN4CIUE 101084054 ”European Lighthouse of AI for Sustainability - ELIAS” Horizon Europe 101120237



(a) NSER IBVS (b) ConvNet

Figure 5. 8 NSER IBVS Con-
vNet

((m) / (s)) ↓	(px) ↓	IoU ↑	((m) / (s)) ↓	(px) ↓	IoU ↑
5.312 / 23.466	29.256	0.530	5.798 / 36.721	30.134	0.620
5.193 / 24.164	13.319	0.759	5.735 / 43.334	28.600	0.6263
5.675 / 24.226	31.800	0.503	5.622 / 41.581	31.499	0.621
6.064 / 28.298	13.172	0.766	5.716 / 45.885	22.802	0.6919
6.196 / 27.315	30.706	0.517	6.493 / 37.535	28.54	0.611
6.041 / 27.917	13.430	0.758	6.490 / 47.238	33.981	0.560
6.846 / 32.358	32.608	0.488	6.197 / 37.166	33.15	0.658
7.043 / 35.535	18.028	0.718	6.316 / 46.363	31.035	0.627
4.177 / 20.228	31.453	0.519	4.559 / 29.921	28.015	0.629
4.089 / 20.481	13.243	0.762	5.065 / 43.841	30.853	0.6482
4.317 / 19.637	31.137	0.494	4.831 / 41.409	32.423	0.612
4.518 / 21.987	13.798	0.759	4.811 / 57.245	43.672	0.5
2.779 / 15.988	28.473	0.518	4.384 / 31.622	28.00	0.611
2.777 / 14.900	13.257	0.763	4.326 / 41.044	39.531	0.5253
2.893 / 13.667	22.618	0.606	4.137 / 33.523	27.89	0.654
3.145 / 17.035	15.839	0.728	3.938 / 38.001	36.195	0.5478
4.774 / 22.111	29.756	0.522	5.253 / 36.185	29.956	0.627
4.859 / 23.790	14.261	0.752	5.300 / 45.369	33.334	0.591

Table 2. - / 4 L2 IoU 3 11×
3

Table 3. 30 1.7M ConvNet 11×

						FPS
NSER IBVS	20.69	7.63	24.56	6.45	82.55	48.30
	1.85	0.93	1.84	1.79	235.64	540.8

References

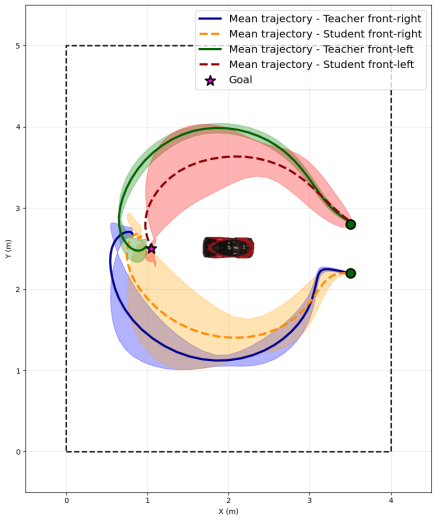


Figure 6. NSER IBVS